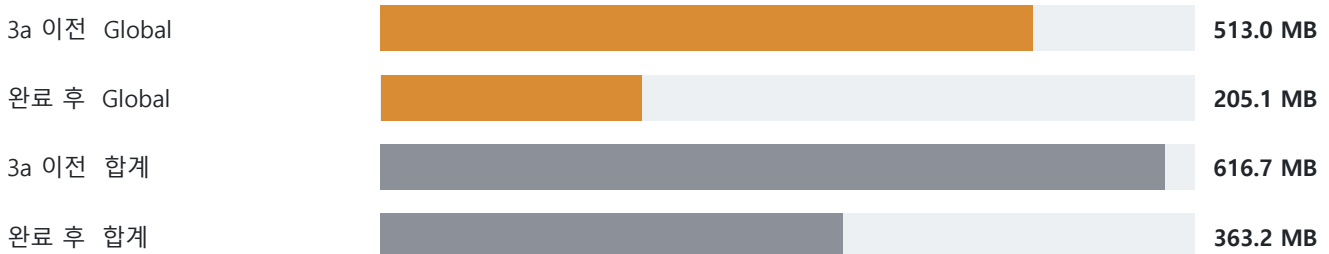


리소스 시스템 Phase 3 — 수명 재배치

2026-07-21 · 에디터 실행(Addressables: Use Existing Build = 실제 번들) · 동일 시나리오 3세션 재현

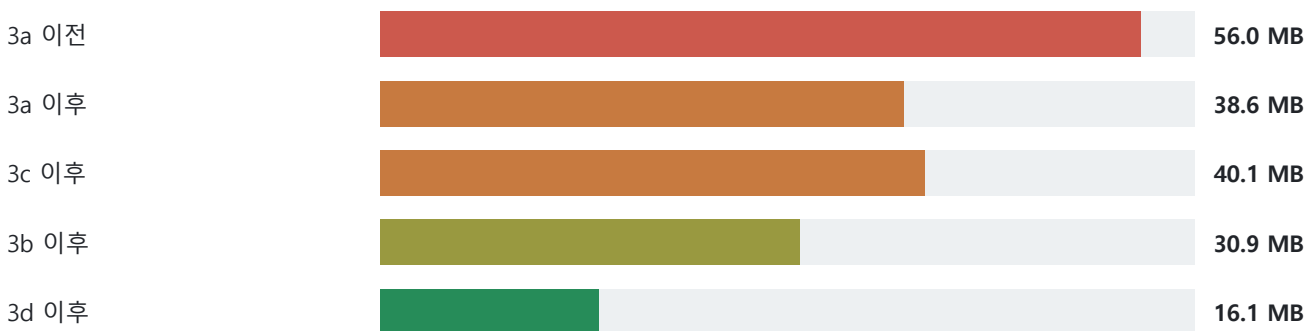
2버킷 시절 "수명이 애매하면 안전하게 isGlobal:true" 관성으로 거의 모든 리소스가 영구 상주하던 것을, 각 리소스의 실제 수명 스코프로 재배치했다. 수동 Release 호출은 한 줄도 추가하지 않았다 — 스코프가 refCount를 자동 정산한다.

진입 직후 메모리 (GameScene, UI 미오픈)



Global -60%, 합계 -41%. 합계까지 줄어든 것은 3c의 라벨 다이어트로 비파티 캐릭터 8종이 애초에 로드되지 않게 됐기 때문이다 (짐을 옮긴 것이 아니라 짐 자체가 사라짐).

세션 전체 Global 단조 증가 — 단계별 추이



-72%. 남은 16.1MB의 구성은 전부 "의도적으로 Global에 둔 것"이다 — DontDestroyOnLoad 토스트, 3씬 공통 상시 UI 2종, exitPopup.

수명 계층

스코프 4종 · Dispose 혹은 위치가 설계의 핵심

Global

부팅 → 게임 종료

자동 · Dispose 안 함 (불변)

- 공용 VFX 5종 · exitPopup · SceneTable
- UI_LootNotification (DontDestroyOnLoad 인스턴스 — R6)
- MonsterHPBar / UnitName / UI_InteractPrompt (3씬 공통 상시)

Scene

씬 진입 → 다음 전환

자동 · ChangeSceneScope

- 맵 · 씬 UI · 팝업 프리팹(3a)
- 아이콘 아틀라스 8종(3d) — 기본 스코프
- 씬 회전마다 전량 반납 → 왕복 후 103.8MB 복귀 확인

Party

파티 구성 → 해체/교체

수동 · GameScene.CreateCharacters

- 인게임 캐릭터 프리팹 (파티원만)
- 씬 전환을 넘어 생존 — 54.3MB 전 구간 불변으로 검증
- 메모리 이득 0 (인스턴스가 프리팹 참조) · 값어치는 회수 경로 확보

Popup

스택 0→1 → 1→0

수동 · ClosePopupUI

- 스탠딩 이미지 · ESC 메뉴 데코
- 중첩 팝업은 스코프 공유 (타입당 1개)
- 재로드 실측: 첫 로드 197ms → 재로드 4ms

Dungeon 스코프는 만들지 않았다. 명세의 목표였던 "던전 왕복 +18.7MB → 0"이 3a/3c로 이미 달성돼 있었다(측정 +0.9 / -0.8MB). 목표가 충족된 상태에서 스코프를 늘리는 것은 순수한 복잡도 증가이므로, 명세를 따르지 않고 측정을 따랐다.

순서 규칙 — Party · Popup 공통

① 인스턴스 파괴 → ② 스코프 회전·Dispose

역순이면 살아있는 인스턴스가 프리팹의 메시·텍스처를 참조하는 동안 핸들만 반납되어, refCount는 풀려도 실제 언로드가 일어나지 않는다.

단계별 기여와 비용

성과와 대가를 나란히

무엇이 무엇을 잡았나

단계	한 일	성과
3a	팝업 프리팝 isGlobal 제거 → Scene	팝업 세션 +56.0 → +10.4MB (-81%). Scene 101→160→101 왕복 복귀로 회수 증명
3c	Party 스코프 + Global 라벨 42→12개	진입 Global 513 → 65.8MB. 합계까지 -254MB (비파티 8종 미로드)
3b	Popup 스코프 신설 (첫 수동 혹)	단조 증가 +40.1 → +30.9MB. 스탠딩·데코 Global에서 소멸
3d	아틀라스 캐시 수명 동행 (R3)	단조 증가 +30.9 → +16.1MB. AtlasSpriteCache 278 고정 해제
R6	LootNotification 수명 불일치 수정	메모리 변화 0 — 회계 정직화 (아래 참조)

대가 — 상시 부담을 일회성 지연으로 바꾼 거래

팝업	3a 이전 요청→표시	3c 이후	비고
UI_Info	2.7 ms	23.2 ms	프리로드로 가려져 있던 실제 로드 비용
UI_Inventory	1.4 ms	15.2 ms	동일
UI_EquipmentUpgradePanel	36.1 ms	36.8 ms	원래 lazy — 대조군, 변화 없음

재오픈은 전 팝업 8.9~10.6ms로 회귀 없음. 재로드 발생 키 18 → 21개(아틀라스가 씬마다 재로드). 즉 진입 -308MB의 대가는 팝업 첫 오픈 +14~20ms이며, 이 트레이드오프를 측정으로 판단한 것이 이 단계의 핵심 활동이다.

R6은 성과가 아니라 비용이었다

① 진입 직후	3c	3b + R6	변화
Global	65.8 MB	206.6 MB	+140.8
Scene	242.3 MB	101.3 MB	-141.0
합계	362.4 MB	362.1 MB	-0.3 (보존)

완벽한 보존 = 같은 바이트가 이름표만 바꿔 달았다. 이전에는 "핸들은 반납됐는데 인스턴스는 살아있는" 유령이라 어느 버킷에도 잡히지 않았다. 메모리가 는 것이 아니라 원래 상주하던 것이 회계에 드러난 것 — "메모리가 줄었다"는 지표가 실은 누수를 세지 못한 결과일 수 있다.

계측이 도구였다

스코프화가 드러낸 기존 버그 3건 + 계측 자체의 버그 2건

아래 셋은 Phase 3가 만든 버그가 아니다. 공통 뿌리는 "에셋의 수명 ≠ 그 에셋으로 만든 인스턴스의 수명"이며, 다버킷 계측을 붙이자 비로소 불일치가 드러났다.

#	버그	정체
5-4 / R6	UI_LootNotification use-after-release	DontDestroyOnLoad 인스턴스 + static 캐시 + Scene 스코프 에셋. 던전 왕복 후 재로드도 안 됨
3a 동반	_isLoadingPopup 미복구	조기 반환 시 플래그가 true로 남아 이후 모든 팝업이 영구 차단
R6 동반	PreloadAsync 영구 정지	로드 실패 시 _instance는 null인데 플래그만 내려가 WaitUntil이 무한 대기

교훈: 스코프를 고를 때 "누가 이 에셋을 쓰는가"가 아니라 "그것으로 만든 것이 언제까지 사는가"를 물어야 한다.

계측 자체의 버그

문제	증상	조치
로드 통계 소실	RecordLoad 호출부가 R1 죽은 코드 제거 때 함께 삭제 → 이후 모든 측정에서 "[LoadStats] 기록 없음"	ResourceRegistry 캐시 미스 경로에 재배포
assert 위치 오류	"이전 썬 미반납"을 잡겠다면서 실제로는 "회전 후 새 적재"를 잡음 (매 전환 오탐)	검사를 await 이전으로 이동 + 취소된 획득 되돌리기

교훈: 죽은 코드를 지울 때 그 안에 살아있는 계측이 없었는지 확인해야 한다. 계측은 기능이 아니라 눈에 띄지 않으므로, 함께 사라져도 즉시 티가 안 나고 몇 단계 뒤 "왜 이 지표가 비지?"로 돌아온다.

측정 설계 — 단일 변수 A/B

Baseline v2는 옛 2버킷 디렉터리로 켜 값인데 그 사이 계측이 레지스트리 기반으로 교체됐다. 도구가 바뀐 구간을 두고 직접 차감하면 성과와 도구가 섞이므로, isGlobal 한 줄만 토글해 같은 도구로 Before를 재측정하고 그 쌍만 비교했다.

부수 소득: 재측정 Before의 진입 512.8 / 팝업직전 515.4MB가 v2의 512.8 / 515.5와 소수점까지 일치 → 도구 증가성이 실측으로 확인되어 기준선 연속성을 확보했다. 숫자를 믿기 전에 그 숫자를 만든 도구를 먼저 검증했다.

최종 성과와 한계

Baseline v2 → Phase 3 완료

지표	3a 이전	완료 후	변화
진입 직후 Global	513.0 MB	205.1 MB	-60%
진입 직후 합계	616.7 MB	363.2 MB	-41%
세션 종료 합계	655.4 MB	379.4 MB	-42%
Global 단조 증가	+56.0 MB	+16.1 MB	-72%
던전 왕복 영구 증가	+18.7 MB	0	목표 달성
AtlasSpriteCache	278 영구 상주	소비자 수명 동행	해소
Scene 왕복 복귀	—	103.8 → 103.8	회귀 없음
수동 Release 호출	—	0 줄	refCount 자동 정산

신뢰도 등급 — 주장 시 반드시 동반

등급	대상	규칙
강	버킷 라벨 전환 · Scene 왕복 복귀 · 로드/재로드 키 목록 · 라벨 42→12	그대로 인용 가능
강 (상향)	증감 · 비율 (-60%, -72% 등)	Play Mode Script가 전 기간 Use Existing Build = 실제 번들 환경임이 확인되어 상향
중	메모리 절대값	에디터는 CPU 사본 포함으로 과대 계상. 개발 빌드 실측 0회 — 절대값 주장 금지
약	히치 ms	n=1~3. 15ms 이하 차이는 주장 금지

미해결 · 남은 작업

항목	내용
UI_Info 히치 이상치	3c 세션에서 563.6ms까지 튜. 이후 103~120ms로 안정(v2 123.4ms 수준)이나 563ms의 원인은 미규명
개발 빌드 실측	계측의 절반([ResourceMemory] 버킷 MB)이 EditorUtility.CollectDependencies 의존 → 런타임 오버레이 선행 필요
전환 중 인플라이트	UI 루트가 DontDestroyOnLoad라 씬 UI가 씬보다 오래 사는 구조 — Phase 4의 몫
R4	NoCache/JSON use-after-release → 임시 스코프 + .text 복사
R5 / Phase 4	풀↔에셋 수명 통합 (PoolManager를 스코프 소속으로)
측정 시나리오 고정	세션마다 팝업 오픈 순서·구성이 달라 히치 등급 상향이 막혀 있음